

Student Performance Prediction Model Deployment

1. Introduction

Machine Learning models are useful only when they can be accessed by users or other applications. Model deployment is the process of making a trained machine learning model available through an application or an API.

In this project, a Random Forest classification model was deployed using Flask. The application was containerized using Docker, making it portable and easy to run on any machine with Docker installed.

2. Objective

The objectives of this project were:

- Train a machine learning model.
 - Save the trained model using Pickle.
 - Build a Flask API for predictions.
 - Containerize the application using Docker.
 - Test the API using Postman.
-

3. Dataset

The Students Performance dataset contains academic information about students.

Features include:

- Gender
- Race/Ethnicity
- Parental Education
- Lunch
- Test Preparation Course
- Math Score
- Reading Score
- Writing Score

A new feature called Average Score was created.

The target variable was:

- Pass (1)
- Fail (0)

4. Model Training

A Random Forest Classifier was trained using Scikit-learn.

The trained model was saved as:

`model.pkl`

using the Pickle library.

5. API Development

A REST API was developed using Flask.

Two endpoints were created.

Home Endpoint

GET /

Displays the API status.

Prediction Endpoint

POST /predict

Accepts student information in JSON format and returns a prediction.

6. Docker Containerization

The application was containerized using Docker.

A Dockerfile was created to:

- Install Python
- Copy project files
- Install dependencies
- Run the Flask application

The Docker image was built using:

```
docker build -t student-api .
```

The Docker container was executed using:

```
docker run -p 5000:5000 student-api
```

7. API Testing

The API was tested using Postman.

A JSON request containing student details was sent to the prediction endpoint.

Example response:

```
{  
  "Prediction": 1  
}
```

This confirmed that the deployed model was functioning correctly.

8. Results

The project successfully achieved the following:

- Machine learning model deployment
 - Flask API creation
 - Docker containerization
 - API testing using Postman
 - Successful prediction generation
-

9. Challenges Faced

During development, several issues were encountered:

- Missing model.pkl file
- HTTP 405 Method Not Allowed
- HTTP 500 Internal Server Error
- Feature name mismatch
- Docker Desktop WSL update requirement
- Missing Dockerfile

Each issue was resolved through debugging and testing.

10. Conclusion

This project demonstrated the complete deployment lifecycle of a machine learning model. Flask was used to expose the model as a REST API, while Docker ensured the application could run consistently across different environments.

The project provides a strong foundation for deploying machine learning applications to cloud platforms such as AWS, Azure, or Google Cloud.